Exceções

Tratamento de Exceções

Agenda

- Compilando e executando com exceções
  - Tipo de exceções
- Tratamentos (try) e Lançamentos (throw)
- Criando Exceções
  - Classes e lançamento

Controlando Erros com Exceções

- O que é uma Exceção (exception)?
  - Uma exceção é um evento que ocorre durante a execução de um programa que indica a ocorrência de algo especial.
  - Uma exceção é resultado da ocorrência de erros durante a execução.
  - Quando ocorre um erro em um método, este método cria um objeto Exception com os dados do erro.
  - O sistema de execução com a Exception tenta encontrar “alguém” para tratar o erro.

Controlando Erros com Exceções

```
class Exception1 {
    public static void main(String args[]) {
        int i=0;
        int j=1/i;
        System.out.println("Não vai funcionar.");
    }
}
```

Controlando Erros com Exceções

```
class Exception1 {
    public static void main(String args[]) {
        int i=0;
        int j=1/i;
        System.out.println("Não vai funcionar.");
    }
}
```

- Este programa gera a saída:

```
java.lang.ArithmeticException: / by zero
     at Exception1.main(exception1.java:4)
```

Controlando Erros com Exceções

```
class Exception1 {
    public static void main(String args[]) {
        int i=0;
        int j=1/i;
        System.out.println("Não vai funcionar.");
    }
}
```

- Este programa gera a saída:

```
java.lang.ArithmeticException: / by zero
    at Exception1.main(exception1.java:4)
```

nome da exceção e o motivo

nome do arquivo e linha onde o erro ocorreu

nome da classe e do método

Exceções

```
class Exception2 {
    static void GeraException() {
        int i=0;
        int j=1/i;
    }

    public static void main(String args[]) {
        GeraException();
        System.out.println("Não vai funcionar.");
    }
}
```

Exceções

- O programa anterior gera a saída:

```
java.lang.ArithmeticException: / by zero
    at Exception2.GeraException(exception2.java:4)
    at Exception2.main(exception2.java:8)
```

- A exceção foi lançada do método GeraException para o main e em seguida para o sistema (não houve tratamento explícito).

Tratando Exceções

tratamento de exceção

```
class Exception3 {
    static void GeraException() {
        try {
            int i = 0;
            int j = 1 / i;
        } catch (ArithmeticException e) {
            System.out.println("Erro aritmetico:\n  " + e);
        }
    }

    public static void main(String args[]) {
        GeraException();
        System.out.println("Vai funcionar.");
    }
}
```

Tratando Exceções

tratamento de exceção

```
class Exception3 {
    static void GeraException() {
        try {
            int i = 0;
            int j = 1 / i;
        } catch (ArithmeticException e) {
            System.out.println("Erro aritmetico:\n  " + e);
        }
    }

    public static void main(String args[]) {
        GeraException();
        System.out.println("Vai funcionar.");
    }
}
```

Saída:
Erro aritmetico:
  java.lang.ArithmeticException: / by zero
Vai funcionar.

Exceções

```
class Exception4 {
    static void GeraException() {
        int i = 0;
        int j = 1 / i;
    }

    public static void main(String args[]) {
        try {
            GeraException();
        } catch (ArithmeticException e) {
            System.out.println("Erro aritmetico:  " + e);
        }
        System.out.println("Vai funcionar.");
    }
}
```

O erro ocorrido em GeraException() foi tratado no main()

Exercício

- Escreva um programa que solicite ao usuário que digite um número inteiro. O programa deve tentar converter a entrada para um número inteiro (int). Se o usuário digitar um valor inválido (exemplo: "abc"), o programa deve capturar a exceção NumberFormatException, exibir uma mensagem de erro amigável e pedir para o usuário digitar novamente.

Tipos de Exceções

- Existem dois tipos de exceções:
  1. descendentes da classe RuntimeException
     - não precisa ser tratada nem lançada, o erro será tratado pelo sistema
  2. descendentes da classe Exception
     - deve ser tratada ou lançada, se não a compilação não terá sucesso. Este tipo de exceção é dito “checked”

Exception

```
import java.io.FileInputStream;
import java.io.File;

class Main {

    public static void main(String[] args) {
        File file = new File("arquivo.dat");
        FileInputStream in = new FileInputStream(file);
        int c;

        while ((c = in.read()) > -1) {
            if (!Character.isSpace((char) c))
                System.out.print((char) c);
        }
        in.close();
        System.out.flush();
    }
}
```

Exception

- Erros do programa anterior (na compilação):

```
Main.java:8: unreported exception java.io.FileNotFoundException; must be caught or declared to be thrown
    FileInputStream in = new FileInputStream(file);
                         ^
Main.java:11: unreported exception java.io.IOException; must be caught or declared to be thrown
    while ((c=in.read()) > -1) {
                ^
Main.java:15: unreported exception java.io.IOException; must be caught or declared to be thrown
    in.close();
      ^
```

Exceções

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        File file = new File(args[0]);
        try {
            FileInputStream in = new FileInputStream(file);
            int c;
            while ((c = in.read()) > -1) {
                if (!Character.isSpace((char) c))
                    System.out.print((char) c);
            }
            in.close();
            System.out.flush();
        } catch (FileNotFoundException e) {
            System.err.println(file + " is not found");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Exceções

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        File file = new File(args[0]);
        try {
            FileInputStream in = new FileInputStream(file);
            int c;
            while ((c = in.read()) > -1) {
                if (!Character.isSpace((char) c))
                    System.out.print((char) c);
            }
            in.close();
            System.out.flush();
        } catch (FileNotFoundException e) {
            System.err.println(file + " is not found");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Exceções

pode lançar uma exceção ao método chamador

```
import java.io.*;

class Exception5 {
    static void GeraException() throws FileNotFoundException {
        File file = new File("arquivo.dat");
        FileInputStream in = new FileInputStream(file);
    }
    public static void main(String args[]) {
        GeraException();
        System.out.println("Não vai funcionar.");
    }
}
```

- Na compilação obteremos:

```
Exception5.java:11: unreported exception java.io.FileNotFoundException; must
be caught or declared to be thrown
    GeraException();
    ^
1 error
```

Exceções - finally

```
class Exception8 {
    static void GeraException() {
        int i = 0;
        int j = 1 / i;
    }

    public static void main(String args[]) {
        try {
            GeraException();
        } finally {
            System.out.println("Vai funcionar sempre.");
        }
        System.out.println("Não vai funcionar.");
    }
}
```

funciona sempre com ou sem exceções

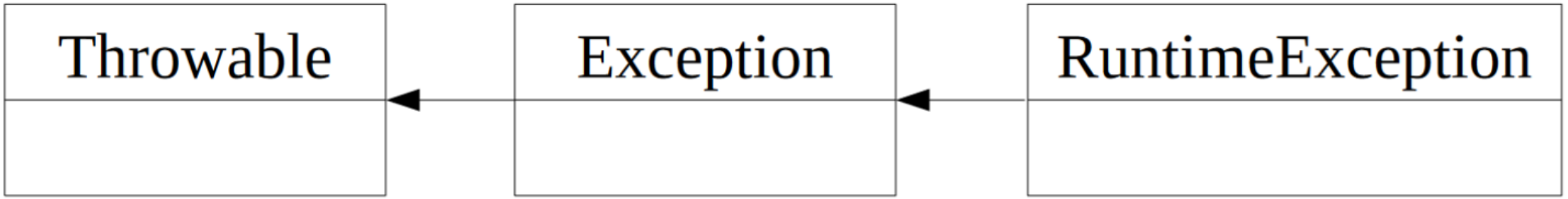
- Este programa gera a saída:

```
Vai funcionar sempre.
java.lang.ArithmeticException: / by zero
     at Exception8.GeraException(exception8.java:4)
     at Exception8.main(exception8.java:9)
```

Criando Exceções

Criando Exceções

- Exceções podem ser criadas com throw para representar erros
- Para enviar as exceções criadas à função chamadora da função atual, lançamos a exceção com throws
- Uma exceção é uma instância de uma classe descendente de Throwable



Exceções

```
class Exception5 {
    static void GeraException() throws Exception {
        throw new Exception("Lancando uma excecao.");
    }

    public static void main(String args[]) {
        GeraException();
        System.out.println("Não vai funcionar.");
    }
}
```

- Na compilação obteremos:

```
exception5.java:7: Exception java.lang.Exception must be caught, or it
must be declared in the throws clause of this method.
    GeraException();
                   ^
1 error
```

Exceções

pode lançar uma exceção ao método chamador

```
class Exception5 {
    static void GeraException() throws Exception {
        throw new Exception("Lancando uma excecao.");
    }

    public static void main(String args[]) {
        GeraException();
        System.out.println("Não vai funcionar.");
    }
}
```

- Na compilação obteremos:

```
exception5.java:7: Exception java.lang.Exception must be caught, or it
must be declared in the throws clause of this method.
    GeraException();
                  ^
1 error
```

Exceções

```
class Exception5 {
    static void GeraException() throws Exception {
        throw new Exception("Lancando uma excecao.");
    }

    public static void main(String args[]) {
        GeraException();
        System.out.println("Não vai funcionar.");
    }
}
```

cria instância da *Exception*

- Na compilação obteremos:

```
exception5.java:7: Exception java.lang.Exception must be caught, or it
must be declared in the throws clause of this method.
    GeraException();
                   ^
1 error
```

Exceções

lança exceção

```
class Exception5 {
    static void GeraException() throws Exception {
        throw new Exception("Lancando uma excecao.");
    }

    public static void main(String args[]) {
        GeraException();
        System.out.println("Não vai funcionar.");
    }
}
```

- Na compilação obteremos:

```
exception5.java:7: Exception java.lang.Exception must be caught, or it
must be declared in the throws clause of this method.
    GeraException();
                  ^
1 error
```

Exceções

```
class Exception6 {
    static void GeraException() throws Exception {
        throw new Exception("Lancando excecao.");
    }

    public static void main(String args[]) throws Exception {
        GeraException();
        System.out.println("Não vai funcionar.");
    }
}
```

- Este programa gera a saída:

```
java.lang.Exception: Lancando excecao.
   at Exception6.GeraException(exception6.java:3)
   at Exception6.main(exception6.java:7)
```

  - Aqui a exceção não é tratada, ela é apenas enviada (lançada) a um nível superior.

Exceções – try - catch

```
class Exception7 {
    static void GeraException() throws Exception {
        throw new Exception("Lancando excecao.");
    }

    public static void main(String args[]) {
        try {
            GeraException();
        } catch (Exception e) {
            System.out.println("Erro: " + e);
        }
        System.out.println("Vai funcionar.");
    }
}
```

- Este programa gera a saída:

```
Erro: java.lang.Exception: Lancando excecao.
Vai funcionar.
```

Criando Novas Exceções

```
class ExcecaoDeTeste extends Exception {
    public String toString() {
        return "ExcecaoDeTeste aconteceu";
    }
}

class Exception9 {
    static void GeraException() throws ExcecaoDeTeste {
        throw new ExcecaoDeTeste();
    }

    public static void main(String args[]) {
        GeraException();
        System.out.println("Não vai funcionar.");
    }
}
```

Na compilação obteremos:

```
exception9.java:13: Exception
ExcecaoDeTeste must be caught, or it
must be declared in the throws
clause of this method.
    GeraException();
                   ^
1 error
```

Exceções

```
class ExcecaoDeTeste extends Exception {
    public String toString() {
        return "ExcecaoDeTeste aconteceu";
    }
}

class Exception10 {
    static void GeraException() throws ExcecaoDeTeste {
        throw new ExcecaoDeTeste();
    }

    public static void main(String args[]) {
        try {
            GeraException();
        } catch (ExcecaoDeTeste e) {
            System.out.println("Tratando Excecao." + e);
        } finally {
            System.out.println("Funciona sempre.");
        }
        System.out.println("Vai funcionar.");
    }
}
```

Este programa gera a saída:

```
Tratando Excecao.ExcecaoDeTeste
aconteceu
Funciona sempre.
Vai funcionar.
```

Exercício

- Crie uma classe ContaBancaria com os atributos saldo e titular. Implemente um método **sacar(double valor)**, que diminui o saldo da conta.
  - Se o saldo for insuficiente, o método deve lançar uma exceção personalizada chamada **SaldoInsuficienteException**, que deve ser criada pelos alunos como uma subclasse de **Exception**.